Assignment for the Full Stack .NET Developer position

Your objective is to build a micro-service for aggregation, retrieval & caching of financial instruments prices.

A simple use case is that a user requests the bitcoin price (BTC/USD) at a specific time-point. The micro-service should fetch the prices from multiple sources, aggregate them using some formula, then return it to the client. Furthermore, it should persist the aggregate price for that time-point so that it's served immediately from the datastore in case it's requested again.

You can use the following two BTC/USD price sources:

1. Bitstamp API endpoint:
   https://www.bitstamp.net/api/#tag/Market-info/operation/GetOHLCData
   example:
   https://www.bitstamp.net/api/v2/ohlc/btcusd/?step=3600&limit=1&start=1672531200
2. Bitfinex API endpoint: https://docs.bitfinex.com/reference/rest-public-candles# example:
   https://api-pub.bitfinex.com/v2/candles/trade:1h:tBTCUSD/hist?start=1672531200000&end=1
   6725 34800000&limit=1

The above APIs request the 1h OHLC candle (open, high, low, close prices during an 1 hour window), by specifying the financial instrument name (btc/usd) and the end of the time window with hour-accuracy. From their response we'll use only the close price field, which is the price at the requested time point.

The aggregation of the multiple prices to a single one can be something simple like the average of them.

Requirements

You will need to provide a working prototype of the application. The requirements are:

- Provide API Endpoint to request the aggregated bitcoin price at a specific time point with hour accuracy.
    - The aggregated price will be served from the datastore if it's already available
    - If not available, the prices will be requested from the external sources, then aggregated, then persisted & returned by the endpoint
- Provide API Endpoint that fetches the persisted bitcoin prices from the datastore during a user-specified time range
- End-points will be exposed as GRPC or RESTful API
- Implementation using .NET 6 or later
- Some options for datastore:
    - In-memory database
    - SQLite
    - Database in docker container (docker-compose file should be provided)
- All prices should be handled as floating-point types
- Time-points of the requested/persisted prices should have hour-accuracy (no minutes/seconds allowed)
- In the future the system may evolve to fetch data from many more sources, or change the price aggregation formula to another more complex one.

Additional requirements for the Full Stack .NET Developer position

In addition to the backend service described above, please also build a simple frontend application that consumes your backend API.

The frontend should allow a user to:

- select or enter the instrument
- select or enter the requested timestamp
- submit the request
- view the aggregated price
- view the source prices used in the calculation
- see whether the result came from cache / datastore or from external API calls
- view a useful error message when something goes wrong

The UI does not need to be visually fancy, but it should be clear and usable.

You may use any frontend technology you are comfortable with. However, our preference is:

1. Blazor
2. React
3. Angular

Other frontend frameworks are also acceptable if the solution is easy to run and review.

An example UI result could look like this:

```
BTC/USD price at 2023-01-01 00:00 UTC

Aggregated price: 16530.25 USD
Result source: External APIs

Source prices:
- Bitstamp: 16531.00
- Bitfinex: 16529.50
```

If the same request is submitted again, the UI could display:

```
BTC/USD price at 2023-01-01 00:00 UTC

Aggregated price: 16530.25 USD
Result source: Cache / datastore

Source prices:
- Bitstamp: 16531.00
- Bitfinex: 16529.50
```

Please include instructions for running both the backend and the frontend.

The repository should also briefly explain:

- the frontend technology used
- how the frontend communicates with the backend
- any assumptions made
- any setup steps required

For the full-stack assignment, we will also look at:

- frontend/backend integration
- basic UI usability
- frontend error handling
- clear project structure
- readability of the frontend code

Your solution will be evaluated based on the requirements listed above and programming best practices.

Do not use an AI generated solution. The purpose of the exercise is to give you an opportunity to show us your engineering skills.